Phase 4 — Benchmark Harness & Load Testing

Goal of this phase: a rigorous, repeatable benchmark that drives real HTTP load at each engine, records throughput and latency percentiles (p50/p90/p99), TTFT, and GPU utilization, and produces the graphs for the README. Validated locally on CPU/MPS first — zero cost — with a tight GPU runbook for later.

---

1. What I built

- **`benchmark/load_test.py`** — an async `httpx` client that fires `num_requests` with a semaphore-bounded **concurrency** cap (closed-loop load), measuring each request's client-side end-to-end latency and reading the server's TTFT/token count.
- **`benchmark/stats.py`** — pure functions: `percentile()` (numpy-style linear interpolation) and `summarize()` (aggregate throughput, p50/p90/p99 latency, mean TTFT). Split out so they're unit-tested deterministically.
- **`benchmark/gpu.py`** — a background `nvidia-smi` sampler that records GPU utilization and memory during a run, and **gracefully no-ops on CPU/MPS** so the same code runs locally and on the cloud GPU.
- **`benchmark/runner.py`** — spawns the server, sweeps `engines × concurrency`, attaches GPU stats, and writes structured `results/bench-<ts>.json`, a flat `.csv`, and a stable `latest.json`.
- **`benchmark/plot.py`** — turns results into clean PNGs: throughput, p50, p99, and TTFT vs concurrency, one line per engine.
- **Tests (`tests/test_benchmark.py`)** — percentile/summary math, plus an in-process load test that drives the real FastAPI app via `httpx.ASGITransport` (no live port needed).

Verification (MPS, CPU validation run): 17/17 tests pass. Sample sweep below.

---

2. Methodology (why these choices)

Choice	Why
Real HTTP load, not in-process calls	Benchmarks must include serialization, queueing, and the server path — that's what a client actually experiences.
Closed-loop concurrency (semaphore)	"N in-flight requests" is the variable that exposes batching: it's the x-axis of every graph. Keeps the comparison apples-to-apples across engines.
Client-side latency for percentiles	Captures time spent *waiting in the queue* under load, which is exactly where naive serving falls apart — server-reported time alone would hide it.
p50/p90/p99, not just mean	Tail latency is what SLAs are written against; a good mean can hide a terrible p99.

Choice	Why
GPU sampler is best-effort + optional	Never let utilization sampling crash a run; absent on CPU/MPS, automatic on GPU.
`ASGITransport` for tests	Drives the true async app in-process — fast, deterministic, no flaky port binding.

---

3. Sample results (CPU/MPS validation — Qwen2.5-0.5B)

Command: `python benchmark/runner.py --concurrency 1,4,16 --requests 16 --max-tokens 32`

engine	concurrency	throughput (tok/s)	p50 (s)	p99 (s)	TTFT (s)
naive	1	46.0	0.57	1.89	0.038
naive	4	54.9	1.86	2.23	0.032
naive	16	55.3	3.57	**7.49**	0.031
batched	1	55.3	0.56	0.59	0.040
batched	4	59.0	1.96	2.44	0.095
batched	16	**79.9**	3.54	**5.11**	1.286

Reading the graphs: - **Throughput:** naive is flat (~55 tok/s, serialized); batched climbs to ~80 — the gap widens with load. This is the headline chart. - **p99 latency:** naive's tail explodes (1.9s → 7.5s) as requests queue behind each other; batched's tail stays lower (5.1s) because it serves them together. - **TTFT tradeoff (honest):** batched TTFT *rises* under burst (1.29s at c=16) because new requests are prefilled serially on admission and wait for a slot. Batching optimizes **throughput**, and can cost **first-token latency** under a burst — a real tradeoff vLLM mitigates with chunked prefill.

These numbers are small because it's a 0.5B model on Apple MPS — the harness is what's being validated. The real, larger gaps come from the GPU runs (Phase 5).

---

4. Running locally first (do this before spending a cent)

```
# Small, fast validation sweep on CPU/MPS — proves the harness works
python benchmark/runner.py --concurrency 1,4 --requests 8 --max-tokens 24
python benchmark/plot.py            # writes results/*.png
```

Keep numbers tiny locally; you're validating the harness, not measuring performance.

---

5. GPU runbook checklist (Phase 5 — budget-aware)

Before benchmarking (confirm you're actually on the GPU): - [ ] `nvidia-smi` prints a GPU — if not, you're on the wrong instance. - [ ] `python -c "import torch; print(torch.cuda.is_available())"` → `True`. - [ ] You're on a **spot/preemptible** instance and a **$200 budget alert** is set.

During (confirm the GPU is being used): - [ ] In a second terminal, `watch -n1 nvidia-smi` — utilization should be **>0** and memory should grow when load runs. 0% util means generation isn't hitting the GPU. - [ ] The harness auto-samples utilization (the `nvidia-smi` sampler turns on automatically) and attaches `util_mean_pct` / `mem_max_mib` to each result row.

Run the real sweep:

```
python benchmark/runner.py --concurrency 1,4,16,64 --requests 64 --max-tokens 128
python benchmark/plot.py
```

After (impossible-to-forget shutdown): - [ ] Copy `results/*.json` and `results/*.png` off the instance. - [ ] **Stop/delete the GPU VM.** - [ ] Confirm in the billing console that **nothing is still running.**

---

6. Interview Q&A

Q: Why measure p99 latency instead of average? A: Averages hide tails. With serialized serving, most requests are fast but the ones stuck behind a long generation are terrible — the p99 captures that. SLAs are written on tail latency because that's what the unluckiest users feel.

Q: Open-loop vs closed-loop load testing? A: Closed-loop fixes the number of in-flight requests (concurrency) and lets throughput emerge — great for "throughput vs concurrency" curves. Open-loop fixes an arrival *rate* regardless of whether the server keeps up — better for finding the saturation point. I use closed-loop because concurrency is the variable that exposes batching; open-loop is the natural extension for saturation testing.

Q: Your batched engine has higher throughput but worse TTFT at high concurrency — explain. A: Throughput and first-token latency are different objectives. My engine prefills new requests serially on admission, so under a burst a request waits for a slot and for its prefill before its first token — raising TTFT even as aggregate throughput rises. Production engines (vLLM) interleave chunked prefill with decode to protect TTFT while keeping throughput.

Q: How do you know the GPU is actually being utilized? A: Sample `nvidia-smi` during the run. If utilization sits near 0%, work isn't reaching the GPU (wrong device, CPU fallback, tiny batch). The harness records mean/peak utilization and peak memory so each result row is self-documenting.

Q: Why client-side latency rather than the server's reported time? A: The client experience includes queueing — time the request spends waiting before the server even starts it. Under load

that queueing *is* the latency story, and server-side timers miss it.

---

7. One-line recall

> *"I built an async load-testing harness that sweeps concurrency across engine variants and records throughput, p50/p90/p99 latency, TTFT, and GPU utilization to JSON/CSV plus plotted graphs. It shows naive throughput flat with an exploding p99 tail, while continuous batching scales throughput — and it exposes the real tradeoff that batching can raise TTFT under burst. Validated on CPU first; one command runs the full GPU sweep."*